

Chapter 6. Inheritance

C# classes support *inheritance*, a popular object-oriented code reuse mechanism. When you write a class, you can optionally specify a base class. Your class will derive from this, meaning that everything in the base class will be present in your class, as well as any members you add.

Classes and class-based record types support only single inheritance (so you can only specify one base class). Interfaces offer a form of multiple inheritance. Value types, including `record struct` types, do not support inheritance at all. One reason for this is that value types are not normally used by reference, which removes one of the main benefits of inheritance: runtime polymorphism. Inheritance is not necessarily incompatible with value-like behavior—some languages manage it—but it often has problems. For example, assigning a value of some derived type into a variable of its base type ends up losing all of the fields that the derived type added, a problem known as *slicing*. C# sidesteps this by restricting inheritance to reference types. When you assign a variable of some derived type into a variable of a base type, you're copying a reference, not the object itself, so the object remains intact. Slicing is an issue only if the base class offers a method that clones the object and doesn't provide a way for derived classes to extend that (or it does, but some derived class neglects to extend it).

Classes specify a base class using the syntax shown in [Example 6-1](#)—the base type appears after a colon that follows the class name. When a class has a primary constructor, that colon and base type appear after the constructor parameter list. This example assumes that a class called `SomeClass` has been defined elsewhere in the project, or one of the libraries it uses.

Example 6-1. Specifying a base class

```
public class Derived : SomeClass
{
}

public class DerivedWithPrimaryConstructor(int value) : SomeClass
{
    public override string ToString() => value.ToString();
}

public class AlsoDerived : SomeClass, IDisposable
{
    public void Dispose() { }
}
```

As you saw in [Chapter 3](#), if the class implements any interfaces, these are also listed after the colon. If you want to derive from a class, and you want to implement interfaces as well, the base class must appear first, as the third class in [Example 6-1](#) illustrates.

You can derive from a class that in turn derives from another class. The `MoreDerived` class in [Example 6-2](#) derives from `Derived`, which in turn derives from `Base`.

Example 6-2. Inheritance chain

```
public class Base
{
}

public class Derived : Base
{
}

public class MoreDerived : Derived
{
}
```

This means that `MoreDerived` technically has multiple base classes: it derives from both `Derived` (directly) and `Base` (indirectly, via `Derived`). This is not multiple inheritance because there is only a single chain of inheritance—any single class derives directly from at most one base class. (All classes derive either directly or indirectly from `object`, which is the default base class if you do not specify one.)

Since a derived class inherits everything the base class has—all its fields, methods, and other members, both public and private—an instance of the derived class can do anything an instance of the base class could do. This is the classic *is a* relationship that inheritance implies in many languages. Any instance of `MoreDerived` is a `Derived` and also a `Base`. C#'s type system recognizes this relationship.

Inheritance and Conversions

C# provides various built-in implicit conversions. In [Chapter 2](#), we saw the conversions for numeric types, but there are also ones for reference types. If some type `D` derives from `B` (either directly or indirectly), then a reference of type `D` can be converted implicitly to a reference of type `B`. This follows from the *is a* relationship I described in the preceding section—any instance of `D` is a `B`. This implicit conversion enables polymorphism: code written to work in terms of `B` will be able to work with any type derived from `B`.

Implicit reference conversions are special. Unlike other conversions, they do not change the value in any way. (The built-in implicit numeric conversions all create a new value from their input, often involving a change of representation. The binary representation of the integer 1 looks different for the `float` and `int` types, for example.) In effect, they convert the interpretation of the reference, rather than converting the reference itself or the object it refers to. As you'll see later in this chapter, there are various places where the CLR will take the availability of an implicit reference conversion into account but will not consider other forms of conversion.

WARNING

A custom implicit conversion between two reference types doesn't count as an implicit reference conversion for these purposes, because a method needs to be invoked to effect such a conversion. The cases in which implicit reference conversions are special rely on the fact that the “conversion” requires no work at runtime.

There is no implicit conversion in the opposite direction—although a variable of type `B` could refer to an object of type `D`, there's no guarantee that it will. There could be any number of types derived from `B`, and a `B` variable could refer to an instance of any of them. Nevertheless, you will sometimes want to attempt to convert a reference from a base type to a derived type, an operation sometimes referred to as a *downcast*. Perhaps you know for a fact that a particular variable holds a reference of a certain type. Or perhaps you're not sure and would like your code to provide additional services for specific types. C# offers three ways to do this.

We can attempt a downcast using the cast syntax. This is the same syntax we use for performing nonimplicit numeric conversions, as [Example 6-3](#) shows.

Example 6-3. Feeling downcast

```
public static void UseAsDerived(Base baseArg)
{
    var d = (Derived) baseArg;

    // ...go on to do something with d
}
```

This conversion is not guaranteed to succeed—that's why we can't use an implicit conversion. If you try this when the `baseArg` argument refers to something that's neither an instance of `Derived` nor something derived from `Derived`, the conversion will fail, throwing an `InvalidCastException`. (Exceptions are described in [Chapter 8](#).)

A cast is therefore appropriate only if you're confident that the object really is of the type you expect, and you would consider it to be an error if it turned out not to be. This is useful when an API accepts an object that it will later give back to you. Many asynchronous APIs do this, because in cases where you launch multiple operations concurrently, you need some way of working out which particular one finished when you get a completion notification (although, as we'll see in later chapters, there are various ways to tackle that problem). Since these APIs don't know what sort of data you'll want to associate with an operation, they usually just take a reference of type `object`, and you would typically use a cast to turn it back into a reference of the required type when the reference is eventually handed back to you.

Sometimes, you will not know for certain whether an object has a particular type. In this case, you can use the `as` operator instead, as shown in [Example 6-4](#). This allows you to attempt a conversion without risking an exception. If the conversion fails, this operator just returns `null`.

Example 6-4. The `as` operator

```
public static void MightUseAsDerived(Base b)
{
    var d = b as Derived;

    if (d != null)
    {
        // ...go on to do something with d
    }
}
```

Although this technique is widely used, the introduction of patterns back in C# 7.0 provided a more succinct alternative. [Example 6-5](#) has the same effect as [Example 6-4](#): the body of the `if` runs only if `b` refers to an instance of `Derived`, in which case it can be accessed through the variable `d`. The `is` keyword here indicates that we want to test `b` against a pattern. In this case we're using a declaration pattern, which performs the same runtime type test as the `as` operator. An expression that applies a pattern with `is` produces a `bool` indicating whether the pattern

matches. We can use this as the `if` statement's condition expression, removing the need to compare with `null`. And since declaration patterns incorporate variable declaration and initialization, the work that needed two statements in [Example 6-4](#) can all be rolled into the `if` statement in [Example 6-5](#).

Example 6-5. The `is` operator with a declaration pattern

```
public static void MightUseAsDerived(Base b)
{
    if (b is Derived d)
    {
        // ...go on to do something with d
    }
}
```

In addition to being more compact, the `is` operator also has the benefit of working in one scenario where `as` does not: you can test whether a reference of type `object` refers to an instance of a value type such as an `int`. (This may seem like a contradiction—how could you have a reference to something that is not a reference type? [Chapter 7](#) will show how this is possible.) The `as` operator wouldn't work because it returns `null` when the instance is not of the specified type, but of course it cannot do that for a value type—there's no such thing as a `null` of type `int`. Since the declaration pattern eliminates the need to test for `null`—we just use the `bool` result that the `is` operator produces—we are free to use value types.

TIP

Occasionally you may want to detect when a particular type is present without needing to perform a conversion. Since `is` can be followed by any pattern, you can use a type pattern, e.g., `is Derived`. This performs the same test as a declaration pattern, without going on to introduce a new variable.

When converting with the techniques just described, you don't necessarily need to specify the exact type. These operations will succeed as long as

an implicit reference conversion exists from the object's real type to the type you're looking for. For example, given the `Base`, `Derived`, and `MoreDerived` types that [Example 6-2](#) defines, suppose you have a variable of type `Base` that currently contains a reference to an instance of `MoreDerived`. Obviously, you could cast the reference to `MoreDerived` (and both `as` and `is` would also succeed for that type), but as you'd probably expect, converting to `Derived` would work too.

These four mechanisms also work for interfaces. When you try to convert a reference to an interface type reference (or test for an interface type with a type pattern), it will succeed if the object referred to implements the relevant interface.

Interface Inheritance

Interfaces support inheritance, but it's not quite the same as class inheritance. The syntax is similar, but as [Example 6-6](#) shows, an interface can specify multiple base interfaces. While .NET offers only single implementation inheritance, this limitation does not apply to interfaces because most of the complications and potential ambiguities that can arise with multiple inheritance do not apply to purely abstract types. The most vexing problems are around handling of fields, which means that even interfaces with default implementations support multiple inheritance, because those don't get to add either fields or public members to the implementing type. (When a class uses a default implementation for a member, that member is accessible only through references of the interface's type.)

Example 6-6. Interface inheritance

```
interface IBase1
{
    void Base1Method();
}

interface IBase2
{
    void Base2Method();
}
```

```

}

interface IBoth : IBase1, IBase2
{
    void Method3();
}

```

Although *interface inheritance* is the official name for this feature, it is a misnomer—whereas derived classes inherit all members from their base, derived interfaces do not. It may appear that they do—given a variable of type `IBoth`, you can invoke the `Base1Method` and `Base2Method` methods defined by its bases. However, the true meaning of interface inheritance is that any type that implements an interface is obliged to implement all inherited interfaces. So a class that implements `IBoth` must also implement `IBase1` and `IBase2`. It's a subtle distinction, especially since C# does not require implementers to list the base interfaces explicitly. The class in [Example 6-7](#) only declares that it implements `IBoth`. However, if you were to use .NET's reflection API to inspect the type definition, you would find that the compiler has added `IBase1` and `IBase2` to the list of interfaces the class implements as well as the explicitly declared `IBoth`.

Example 6-7. Implementing a derived interface

```

public class Impl : IBoth
{
    public void Base1Method()
    {
    }

    public void Base2Method()
    {
    }

    public void Method3()
    {
    }
}

```

Since implementations of a derived interface must implement all base interfaces, C# lets you access bases' members directly through a reference

of a derived type, so a variable of type `IBoth` provides access to `Base1Method` and `Base2Method`, as well as that interface's own `Method3`. Implicit reference conversions exist from derived interface types to their bases. For example, a reference of type `IBoth` can be assigned to variables of type `IBase1` and `IBase2`.

Generics

If you derive from a generic class, you must supply the type arguments it requires. If your derived type is also generic, it can use its own type parameters as arguments if you wish, as long as they meet any constraints the base class defines. [Example 6-8](#) shows both techniques and also illustrates that when deriving from a class with multiple type parameters, you can use a mixture, specifying one type argument directly and punting on another. (By the way, I've used C# 11.0's new `required` keyword here because otherwise the compiler warns of possible nullable reference problems: if I constructed a `GenericBase<string>` without initializing `Item`, that property's value would be `null`, despite its type being `string`.)

Example 6-8. Deriving from a generic base class

```
public class GenericBase1<T>
{
    public required T Item { get; set; }
}

public class GenericBase2<TKey, TValue>
    where TValue : class
{
    public required TKey Key { get; set; }
    public required TValue Value { get; set; }
}

public class NonGenericDerived : GenericBase1<string>
{
}

public class GenericDerived<T> : GenericBase1<T>
{
}
```

```

}

public class MixedDerived<T> : GenericBase2<string, T>
    where T : class
{
}

```

Although you are free to use any of your type parameters as type arguments for a base class, you cannot derive from a type parameter. This is a little disappointing if you are used to languages that permit such things, but the C# language specification simply forbids it. However, you are allowed to use your own type as a type argument to your base class. And you can also specify a constraint on a type argument that requires it to derive from your own type. [Example 6-9](#) shows each of these.

Example 6-9. Self-referential type arguments

```

public class SelfAsTypeArgument : IComparable<SelfAsTypeArgument>
{
    // ...implementation removed for clarity
}

public class Curious<T>
    where T : Curious<T>
{
}

```

As you saw in [Chapter 4](#), the generic math interfaces use this kind of constraint. It means that the type `INumber<SomeType>` is valid only if `SomeType` does in fact implement `INumber<SomeType>` (either directly or through inheritance).

Covariance and Contravariance

In [Chapter 4](#), I mentioned that generic types have special rules for type compatibility, referred to as *covariance* and *contravariance*. These rules determine whether references of certain generic types are implicitly con-

vertible to one another when implicit conversions exist between their type arguments.

NOTE

Covariance and contravariance are applicable only to the generic type arguments of interfaces and delegates. (Delegates are described in [Chapter 9](#).) You cannot define a covariant or contravariant class, struct, or record.

Consider the simple `Base` and `Derived` classes shown earlier in [Example 6-2](#), and look at the method in [Example 6-10](#), which accepts any `Base`. (It does nothing with it, but that's not relevant here—what matters is what its signature says it can use.)

Example 6-10. A method accepting any `Base`

```
public static void UseBase(Base b)
{
}
```

We already know that as well as accepting a reference to any `Base`, this can also accept a reference to an instance of any type derived from `Base`, such as `Derived`. Bearing that in mind, consider the method in [Example 6-11](#).

Example 6-11. A method accepting any `IEnumerable<Base>`

```
public static void AllYourBase(IEnumerable<Base> bases)
{
}
```

This requires an object that implements the `IEnumerable<T>` generic interface described in [Chapter 5](#), where `T` is `Base`. What would you expect to happen if we attempted to pass an object that did not implement `IEnumerable<Base>` but did implement `IEnumerable<Derived>`?

[Example 6-12](#) does this, and it compiles just fine.

Example 6-12. Passing an `IEnumerable<T>` of a derived type

```
IEnumerable<Derived> derivedItems = new[] { new Derived(), new Derived() };
AllYourBase(derivedItems);
```

Intuitively, this makes sense. The `AllYourBase` method expects its argument to supply a sequence of objects that are all of type `Base`. An `IEnumerable<Derived>` fits the bill because it supplies a sequence of `Derived` objects, and any `Derived` object is also a `Base`. However, what about the code in [Example 6-13](#)?

Example 6-13. A method accepting any `ICollection<Base>`

```
public static void AddBase(ICollection<Base> bases)
{
    bases.Add(new Base());
}
```

Recall from [Chapter 5](#) that `ICollection<T>` derives from `IEnumerable<T>`, and it adds the ability to modify the collection in certain ways. This particular method exploits that by adding a new `Base` object to the collection. That would mean trouble for the code in [Example 6-14](#).

Example 6-14. Error: trying to pass an `ICollection<T>` with a derived type

```
ICollection<Derived> derivedList = new List<Derived>();
AddBase(derivedList); // Will not compile
```

Code that uses the `derivedList` variable will expect every object in that list to be of type `Derived` (or something derived from it, such as the `MoreDerived` class from [Example 6-2](#)). But the `AddBase` method in [Example 6-13](#) attempts to add a plain `Base` instance. That cannot be correct, and the compiler does not allow it. The call to `AddBase` will produce a compiler error complaining that references of type

`ICollection<Derived>` cannot be converted implicitly to references of type `ICollection<Base>`.

How does the compiler know that it's not OK to do this, while the very similar-looking conversion from `IEnumerable<Derived>` to `IEnumerable<Base>` is allowed? It's not because [Example 6-13](#) contains code that would cause a problem, by the way. You'd get the same compiler error even if the `AddBase` method were completely empty. The reason we don't get an error in [Example 6-12](#) is that the `IEnumerable<T>` interface declares its type argument `T` as covariant. You saw the syntax for this in [Chapter 5](#), but I didn't draw attention to it, so [Example 6-15](#) shows the relevant part from that interface's definition again.

Example 6-15. Covariant type parameter

```
public interface IEnumerable<out T> : IEnumerable
```

That `out` keyword does the job. (Again, C# keeps up the C-family tradition of giving each keyword multiple jobs—we first saw this keyword in the context of method parameters that can return information to the caller.) Intuitively, describing the type argument `T` as “out” makes sense, in that the `IEnumerable<T>` interface only ever *provides* a `T`—it does not define any members that *accept* a `T`. (The interface uses this type parameter in just one place: its read-only `Current` property.)

Compare that with `ICollection<T>`. This derives from `IEnumerable<T>`, so clearly it's possible to get a `T` out of it, but it's also possible to pass a `T` into its `Add` method. So `ICollection<T>` cannot annotate its type argument with `out`. (If you were to try to write your own similar interface, the compiler would produce an error if you declared the type argument as being covariant. Rather than just taking your word for it, it checks to make sure you really can't pass a `T` in anywhere.)

The compiler rejects the code in [Example 6-14](#) because `T` is not covariant in `ICollection<T>`. This usage of the terms *covariant* and *contravariant* come from a branch of mathematics called *category theory*. The parameters that behave like `IEnumerable<T>`'s `T` are called covariant because

implicit reference conversions for the generic type work in the same direction as conversions for the type argument: `Derived` is implicitly convertible to `Base`, and since `T` is covariant in `IEnumerable<T>`, `IEnumerable<Derived>` is implicitly convertible to `IEnumerable<Base>`.

Contravariance works the other way around, and as you might guess, we denote it with the `in` keyword. It's easiest to see this in action with code that uses members of types, so [Example 6-16](#) shows a marginally more interesting pair of classes than the earlier examples.

Example 6-16. Class hierarchy with actual members

```
public class Shape
{
    public required Rect BoundingBox { get; set; }
}

public class RoundedRectangle : Shape
{
    public required double CornerRadius { get; set; }
}
```

[Example 6-17](#) defines two classes that use these shape types. Both implement `IComparer<T>`, which I introduced in [Chapter 4](#). The `BoxAreaComparer` compares two shapes based on the area of their bounding box—the shape whose bounding box covers the greater area will be deemed the larger by this comparison. The `CornerSharpnessComparer`, on the other hand, compares rounded rectangles by looking at how pointy their corners are.

Example 6-17. Comparing shapes

```
public class BoxAreaComparer : IComparer<Shape>
{
    public int Compare(Shape? x, Shape? y)
    {
        if (x is null)
        {
            return y is null ? 0 : -1;
        }
    }
}
```

```

    }
    if (y is null)
    {
        return 1;
    }

    double xArea = x.BoundingBox.Width * x.BoundingBox.Height;
    double yArea = y.BoundingBox.Width * y.BoundingBox.Height;

    return Math.Sign(xArea - yArea);
}
}

public class CornerSharpnessComparer : IComparer<RoundedRectangle>
{
    public int Compare(RoundedRectangle? x, RoundedRectangle? y)
    {
        if (x is null)
        {
            return y is null ? 0 : -1;
        }
        if (y is null)
        {
            return 1;
        }

        // Smaller corners are sharper, so smaller radius is "greater" for
        // the purpose of this comparison, hence the backward subtraction.
        return Math.Sign(y.CornerRadius - x.CornerRadius);
    }
}

```

References of type `RoundedRectangle` are implicitly convertible to `Shape`, so what about `IComparer<T>`? Our `BoxAreaComparer` can compare any shapes and declares this by implementing `IComparer<Shape>`. The comparer's type argument `T` is only ever used in the `Compare` method, and that is happy to be passed any `Shape`. It will not be fazed if we pass it a pair of `RoundedRectangle` references, so our class is a perfectly adequate `IComparer<RoundedRectangle>`. An implicit conversion from `IComparer<Shape>` to `IComparer<RoundedRectangle>` therefore makes sense, and is in fact allowed. However, the

`CornerSharpnessComparer` is fussier. It uses the `CornerRadius` property, which is available only on rounded rectangles, not on any old `Shape`. Therefore, no implicit conversion exists from `IComparer<RoundedRectangle>` to `IComparer<Shape>`.

This is the reverse of what we saw with `IEnumerable<T>`. Implicit conversion is available from `IEnumerable<T1>` to `IEnumerable<T2>` when an implicit reference conversion from `T1` to `T2` exists. But implicit conversion from `IComparer<T1>` to `IComparer<T2>` is available when an implicit reference conversion exists in the other direction: from `T2` to `T1`. That reversed relationship is called contravariance. [Example 6-18](#) is an excerpt of the definition for `IComparer<T>` showing this contravariant type parameter.

Example 6-18. Contravariant type parameter

```
public interface IComparer<in T>
```

Most generic type parameters are neither covariant nor contravariant. (They are *invariant*.) `ICollection<T>` cannot be variant, because it contains some members that accept a `T` and some that return one. An `ICollection<Shape>` might contain shapes that are not `RoundedRectangles`, so you cannot pass it to a method expecting an `ICollection<RoundedRectangle>`, because such a method would expect every object it retrieves from the collection to be a rounded rectangle. Conversely, an `ICollection<RoundedRectangle>` cannot be expected to allow shapes other than rounded rectangles to be added, and so you cannot pass an `ICollection<RoundedRectangle>` to a method that expects an `ICollection<Shape>` because that method may try to add other kinds of shapes.

Arrays are covariant, just like `IEnumerable<T>`. This is rather odd, because we can write methods like the one in [Example 6-19](#).

Example 6-19. Changing an element in an array

```
public static void UseBaseArray(Base[] bases)
{
    bases[0] = new Base();
}
```

If I were to call this with the code in [Example 6-20](#), I would be making the same mistake as I did in [Example 6-14](#), where I attempted to pass an `ICollection<Derived>` to a method that wanted to put something that was not `Derived` into the collection. But while [Example 6-14](#) does not compile, [Example 6-20](#) does, due to the surprising covariance of arrays.

Example 6-20. Passing an array with derived element type

```
Derived[] derivedBases = [new Derived(), new Derived()];
UseBaseArray(derivedBases);
```

This makes it look as though we could sneakily make this array accept a reference to an object that is not an instance of the array's element type—in this case, putting a reference to a non-`Derived` object, `Base`, in `Derived[]`. But that would be a violation of the type system. Does this mean the sky is falling?

In fact, C# correctly forbids such a violation, but it relies on the CLR to enforce this at runtime. Although a reference to an array of type `Derived[]` can be implicitly converted to a reference of type `Base[]`, any attempt to set an array element in a way that is inconsistent with the type system will throw an `ArrayTypeMismatchException`. So [Example 6-19](#) would throw that exception when it tried to assign a reference to a `Base` into the `Derived[]` array.

The runtime check ensures that type safety is maintained, and this enables a convenient feature. If we write a method that takes an array and only reads from it, we can pass arrays of some derived element type. The downside is that the CLR has to do extra work at runtime when you modify array elements to ensure that there is no type mismatch. It may be

able to optimize the code to avoid having to check every single assignment, but there is still some overhead, meaning that arrays are not quite as efficient as they might be.

This somewhat peculiar arrangement dates back to the time before .NET had formalized concepts of covariance and contravariance—these came in with generics, which were introduced in .NET 2.0. Perhaps if generics had been around from the start, arrays would be less odd, although having said that, even after .NET 2.0 for many years the runtime libraries did not provide any other way to pass a collection covariantly to a method that wanted to read from it using indexing. Until .NET Framework 4.5 introduced `ReadOnlyList<T>` (for which `T` is covariant), there was no read-only indexed collection interface in the framework, and therefore no standard indexed collection interface with a covariant type parameter. (`IList<T>` is read/write, so just like `ICollection<T>`, it cannot offer variance.)

While we are on the subject of type compatibility and the implicit reference conversions that inheritance makes available, there is one more type we should look at: `object`.

System.Object

The `System.Object` type, or `object`, as we usually call it in C#, is useful because it can act as a sort of universal container: a variable of this type can hold a reference to almost anything. I've mentioned this before, but I haven't yet explained why it's true. The reason this works is that almost everything derives from `object`.

If you do not specify a base class when writing a class or record, the C# compiler automatically uses `object` as the base. As we'll see shortly, it chooses different bases for certain kinds of types such as structs, but even those derive from `object` indirectly. (As ever, pointer types are an exception—these do not derive from `object`.)

The relationship between interfaces and objects is slightly more subtle. Interfaces do not derive from `object`, because an interface can specify

only other interfaces as its bases. However, a reference of any interface type is implicitly convertible to a reference of type `object`. This conversion will always be valid, because all types that are capable of implementing interfaces ultimately derive from `object`. Moreover, C# chooses to make the `object` class's members available through interface references even though they are not, strictly speaking, members of the interface. This means that references of any kind always offer the following methods defined by `object`: `ToString`, `Equals`, `GetHashCode`, and `GetType`.

The Ubiquitous Methods of `System.Object`

I've mentioned `ToString` a few times already. The default implementation returns the object's type name, but many types provide their own implementation of `ToString`, returning a more useful textual representation of the object's current value. The numeric types return a decimal representation of their value, for example, while `bool` returns either `"True"` or `"False"`.

I discussed `Equals` and `GetHashCode` in [Chapter 3](#), but I'll provide a quick recap here. `Equals` allows an object to be compared with any other object. The default implementation just performs an identity comparison—that is, it returns `true` only when an object is compared with itself. Many types provide an `Equals` method that performs value-like comparison—for example, two distinct `string` objects may contain identical text, in which case they will report being equal to each other. (Should you need to perform an identity-based comparison of objects that provide value-based comparison, you can use the `object` class's static `ReferenceEquals` method.) Incidentally, `object` also defines a static version of `Equals` that takes two arguments. This checks whether the arguments are `null`, returning `true` if both are `null` and `false` if only one is `null`; otherwise, it defers to the first argument's `Equals` method. And, as discussed in [Chapter 3](#), `GetHashCode` returns an integer that is a reduced representation of the object's value, which is used by hash-based mechanisms such as the `Dictionary<TKey, TValue>` collection class.

Any pair of objects for which `Equals` returns `true` must return the same hash codes.

The `GetType` method provides a way to discover things about the object's type. It returns a reference of type `Type`. That's part of the reflection API, which is the subject of [Chapter 13](#).

Besides these public members, available through any reference, `object` defines two more members that are not universally accessible. An object has access to these members only on itself. They are `Finalize` and `MemberwiseClone`. The CLR calls the `Finalize` method to notify you that your object is no longer in use and the memory it occupies is about to be reclaimed. In C# we do not normally work directly with the `Finalize` method, because C# presents this mechanism through destructors, as I'll show in [Chapter 7](#). `MemberwiseClone` creates a new instance of the same type as your object, initialized with copies of all of your object's fields. If you need a way to create a clone of an object, this may be easier than writing code that copies all the contents across by hand, although it is not very fast.

The reason these last two methods are available only from inside the object is that you might not want other people cloning your object, and it would be unhelpful if external code could call the `Finalize` method, fooling your object into thinking that it was about to be freed if in fact it wasn't. The `object` class limits the accessibility of these members. But they're not private—that would mean that only the `object` class itself could access them, because private members are not visible even to derived classes. Instead, `object` makes these members *protected*, an accessibility specifier designed for inheritance scenarios.

Accessibility and Inheritance

By now, you will already be familiar with most of the accessibility levels available for types and their members. Elements marked as `public` are available to all, `private` members are accessible only from within the type that declared them, and `internal` members are available to code

defined in the same component.¹ But with inheritance, we get three other accessibility options.

A member marked as `protected` is available inside the type that defined it and also inside any derived types. But for code using an instance of your type, `protected` members are not accessible, just like `private` members.

The next protection level for type members is `protected internal`. (You can write `internal protected` if you prefer; the order makes no difference.) This makes the member more accessible than either `protected` or `internal` on its own: the member will be accessible to all derived types *and* to all code that shares an assembly.

The third protection level that inheritance adds is `protected private`. Members marked with this (or the equivalent `private protected`) are available only to types that are both derived from *and* defined in the same component (or a friend assembly) as the defining type.

You can use `protected`, `protected internal`, or `protected private` for any member of a type, and not just methods. You can even define nested types with these accessibility specifiers.

While `protected` and `protected internal` (although not `protected private`) members are not available through an ordinary variable of the defining type, they are still part of the type's public API, in the sense that anyone who has access to your classes will be able to use these members. As with most languages that support a similar mechanism, `protected` members in C# are typically used to provide services that derived classes might find useful. If you write a `public` class that supports inheritance, then anyone can derive from it and gain access to its `protected` members. Removing or changing `protected` members would therefore risk breaking code that depends on your class just as surely as removing or changing `public` members would.

When you derive from a class, you cannot make your class more visible than its base. If you derive from an `internal` class, for example, you cannot declare your class to be `public`. Your base class forms part of your

class's API, so anyone wishing to use your class will also in effect be using its base class; this means that if the base is inaccessible, your class will also be inaccessible, which is why C# does not permit a class to be more visible than its base. If you derive from a `protected` nested class, your derived class could be `protected`, `private`, or `protected private` but not `public`, `internal`, or `protected internal`.

NOTE

This restriction does not apply to the interfaces you implement. A `public` class is free to implement `internal` or `private` interfaces. However, it does apply to an interface's bases: a `public` interface cannot derive from an `internal` interface.

When defining methods, there's another keyword you can add for the benefit of derived types: `virtual`.

Virtual Methods

A *virtual method* is one that a derived type can replace. Several of the methods defined by `object` are virtual: the `ToString`, `Equals`, `GetHashCode`, and `Finalize` methods are all designed to be replaced. The code required to produce a useful textual representation of an object's value will differ considerably from one type to another, as will the logic required to determine equality and produce a hash code. Types typically define a finalizer only if they need to do some specialized cleanup work when they go out of use.

Not all methods are virtual. In fact, C# makes methods nonvirtual by default. The `object` class's `GetType` method is not virtual, so you can always trust the information it returns to you because you know that you're calling the `GetType` method supplied by .NET, and not some type-specific substitute designed to fool you. To declare that a method should be virtual, use the `virtual` keyword, as [Example 6-21](#) shows.

Example 6-21. A class with a virtual method

```
public class BaseWithVirtual
{
    public virtual void ShowMessage()
    {
        Console.WriteLine("Hello from BaseWithVirtual");
    }
}
```

NOTE

You can also apply the `virtual` keyword to properties. Properties are just methods under the covers, so this has the effect of making the accessor methods virtual. The same is true for events, which are discussed in [Chapter 9](#).

There's nothing unusual about the syntax for invoking a virtual method. As [Example 6-22](#) shows, it looks just like calling any other method.

Example 6-22. Using a nonstatic virtual method

```
public static void CallVirtualMethod(BaseWithVirtual o)
{
    o.ShowMessage();
}
```

The difference between virtual and nonvirtual instance method invocations is that a virtual method call decides at runtime which method to invoke. The code in [Example 6-22](#) will, in effect, inspect the object passed in, and if the object's type supplies its own implementation of `ShowMessage`, it will call that instead of the one defined in `BaseWithVirtual`. The method is chosen based on the actual type the target object turns out to have at runtime, and not the static type (determined at compile time) of the expression that refers to the target object.

Derived types are not obliged to replace virtual methods. [Example 6-23](#) shows two classes that derive from the one in [Example 6-21](#). The first leaves the base class's implementation of `ShowMessage` in place. The second overrides it. Note the `override` keyword—C# requires us to state explicitly that we are intending to override a nonstatic virtual method.

Example 6-23. Overriding virtual methods

```
public class DeriveWithoutOverride : BaseWithVirtual
{
}

public class DeriveAndOverride : BaseWithVirtual
{
    public override void ShowMessage()
    {
        Console.WriteLine("This is an override");
    }
}
```

We can use these types with the method in [Example 6-22](#). [Example 6-24](#) calls it three times, passing in a different type of object each time.

Example 6-24. Exploiting virtual methods

```
CallVirtualMethod(new BaseWithVirtual());
CallVirtualMethod(new DeriveWithoutOverride());
CallVirtualMethod(new DeriveAndOverride());
```

This produces the following output:

```
Hello from BaseWithVirtual
Hello from BaseWithVirtual
This is an override
```

Obviously, when we pass an instance of the base class, we get the output from the base class's `ShowMessage` method. We also get that with the derived class that has not supplied an override. It is only the final class,

which overrides the method, that produces different output. This shows that virtual methods provide a way to write *polymorphic* code:

[Example 6-22](#) can use a variety of types.

When overriding a method, the method name and its parameter types must be an exact match. In most cases, the return type will also be identical, but it doesn't always need to be. If the `virtual` method's return type is not `void`, and is not a `ref` return, the overriding method may have a different type as long as an implicit reference conversion from that type to the `virtual` method's return type exists. To put that more informally, an override is allowed to be more specific about its return type. This means that examples such as [Example 6-25](#) are legal.

Example 6-25. An override that narrows the return type

```
public class Product { }
public class Book : Product { }

public class ProductSourceBase
{
    public virtual Product Get() { return new Product(); }
}

public class BookSource : ProductSourceBase
{
    public override Book Get() { return new Book(); }
}
```

The return type of the override of `Get` is `Book`, even though the `virtual` method it overrides returns a `Product`. This is fine because anything that invokes this method through a reference of type `ProductSourceBase` will expect to get back a reference of type `Product`, and thanks to inheritance, a `Book` is a `Product`. So users of the `ProductSourceBase` type will be unaware of and unaffected by the change. This feature can sometimes be useful in cases where code working directly with a derived type needs to know the specific type that will be returned.

You might be wondering why we need virtual methods, given that interfaces also enable polymorphic code, but virtual methods do have some advantages. A default interface member implementation cannot define or access nonstatic fields, so it is somewhat limited compared to a class that defines a virtual function. (And since default interface implementations require runtime support, they are unavailable to code that needs to be able to run on .NET Framework, which includes any library targeting .NET Standard 2.0 or older.) However, there is a more subtle advantage available to virtual methods, but before we can look at it, we need to explore a feature of virtual methods that at first glance even more closely resembles the way interfaces work.

Abstract Methods

You can define a virtual method without providing a default implementation. C# calls this an *abstract method*. If a class contains one or more abstract methods, the class is incomplete, because it doesn't provide all of the methods it defines. Classes of this kind are also described as being abstract, and it is not possible to construct instances of an abstract class; attempting to use the `new` operator with an abstract class will cause a compiler error. Sometimes when discussing classes, it's useful to make clear that some particular class is *not* abstract, for which we normally use the term *concrete class*.

If you derive from an abstract class, then unless you provide implementations for all the abstract methods, your derived class will also be abstract. You must state your intention to write an abstract class with the `abstract` keyword; if this is absent from a class that has unimplemented abstract methods (either ones it has defined itself or ones it has inherited from its base class), the C# compiler will report an error. [Example 6-26](#) shows an abstract class that defines a single abstract method. Abstract methods are virtual by definition; there wouldn't be much use in defining a method that has no body if there were no way for derived classes to supply a body.

Example 6-26. An abstract class

```
public abstract class AbstractBase
{
    public abstract void ShowMessage();
}
```

Abstract method declarations just define the signature and do not contain a body. Unlike with interfaces, each abstract member has its own accessibility—you can declare abstract methods as `public`, `internal`, `protected internal`, `protected private`, or `protected`. (It makes no sense to make an abstract or virtual method `private`, because the method will be inaccessible to derived types and therefore impossible to override.)

NOTE

Although classes that contain abstract methods are required to be abstract, the converse is not true. It is legal to define a class as abstract even if it would be a viable concrete class. This prevents the class from being constructed. A class that derives from this will be concrete without needing to override any abstract methods.

Abstract classes have the option to declare that they implement an interface without needing to provide a full implementation. You can't just omit the unimplemented members, though. You must explicitly declare all of its members, marking any that you want to leave unimplemented as being abstract, as [Example 6-27](#) shows. This forces concrete derived types to supply the implementation.

Example 6-27. Abstract interface implementation

```
public abstract class MustBeComparable : IComparable<string>
{
    public abstract int CompareTo(string? other);
}
```

There's clearly some overlap between abstract classes and interfaces. Both provide a way to define an abstract type that code can use without needing to know the exact type that will be supplied at runtime. Each option has its pros and cons. Interfaces have the advantage that a single type can implement multiple interfaces, whereas a class gets to specify only a single base class. But abstract classes can define fields and can use these in any default member implementations they supply. However, there's a more subtle advantage available to virtual methods that comes into play when you release multiple versions of a library over time.

Inheritance and Library Versioning

Imagine what would happen if you had written and released a library that defined some public interfaces and abstract classes, and in the second release of the library, you decided that you wanted to add some new members to one of the interfaces. It's conceivable that this might not cause a problem for customers using your code. Certainly, any place where they use a reference of that interface type will be unaffected by the addition of new features. However, what if some of your customers have written types that implement your interface? Suppose, for example, that in a future version of .NET, Microsoft decided to add a new member to the `IEnumerable<T>` interface.

If the interface were not to supply a default implementation for the new member, it would be a disaster. This interface is widely used but also widely implemented. Classes that already implement `IEnumerable<T>` would become invalid because they would not provide this new member, so old code would fail to compile, and code already compiled would throw `MissingMethodException` errors at runtime. C#'s support for default member implementations in interfaces mitigates this: in the unlikely event that Microsoft did add a new member to `IEnumerable<T>`, it could supply a default implementation preventing these errors. This doesn't help anyone using .NET Framework, which does not support this feature, but for newer runtimes, it makes modification of existing interface definitions seem viable. However, there's a more subtle problem. Some classes might by chance already have had a member with the same name and signature as the newly added method. If that code is recompiled against

the new interface definition, the compiler would treat that existing member as part of the implementation of the interface, even though the developer who wrote the method did not write it with that intention. So unless the existing code coincidentally happens to do exactly what the new member requires, we'd have a problem, and we wouldn't get compiler errors or warnings to alert us.

Consequently, the widely accepted rule is that you do not alter interfaces once they have been published. If you have complete control over all of the code that uses and implements an interface, you can get away with modifying the interface, because you can make any necessary modifications to the affected code. But once the interface has become available for use in codebases you do not control—that is, once it has been published—it's no longer possible to change it without risking breaking someone else's code. Default interface implementations mitigate this risk, but they cannot eliminate the problem of existing methods accidentally being misinterpreted when they get recompiled against the updated interface.

Abstract base classes do not have to suffer from this problem. Obviously, introducing new abstract members would cause exactly the same `MissingMethodException` failures, but introducing new virtual methods does not.

But what if, after releasing version 1.0 of a component, you add a new virtual method in version 1.1 that turns out to have the same name and signature as a method that one of your customers happens to have added in a derived class? Perhaps in version 1.0, your component defines the rather uninteresting base class shown in [Example 6-28](#).

Example 6-28. Base type version 1.0

```
public class LibraryBase
{
}
```

If you release this library, perhaps on the [NuGet package management website](#), or maybe as part of some Software Development Kit (SDK) for

your application, a customer might write a derived type such as the one in [Example 6-29](#). The `Start` method they have written is clearly not meant to override anything in the base class.

Example 6-29. Class derived from version 1.0 base

```
public class CustomerDerived : LibraryBase
{
    public void Start()
    {
        Console.WriteLine("Derived type's Start method");
    }
}
```

Since you won't necessarily get to see every line of code that your customers write, you might be unaware of this `Start` method. So in version 1.1 of your component, you might decide to add a new virtual method, also called `Start`, as [Example 6-30](#) shows.

Example 6-30. Base type version 1.1

```
public class LibraryBase
{
    public virtual void Start() { }
}
```

Imagine that your system calls this method as part of an initialization procedure introduced in v1.1. You've defined a default empty implementation so that types derived from `LibraryBase` that don't need to take part in that procedure don't have to do anything. Types that wish to participate will override this method. But what happens with the class in [Example 6-29](#)? Clearly the developer who wrote that did not intend to participate in your new initialization mechanism, because that didn't exist when they wrote their code. It could be bad if your code calls the `CustomerDerived` class's `Start` method, because the developer presumably expects it to be called only when their code decides to call it. Fortunately, the compiler will detect this problem. If the customer at-

tempts to compile [Example 6-29](#) against version 1.1 of your library ([Example 6-30](#)), the compiler will warn them that something is not right:

```
warning CS0114: 'CustomerDerived.Start()' hides inherited member  
'LibraryBase.Start()'. To make the current member override that implementation,  
add the override keyword. Otherwise add the new keyword.
```

This is why the C# compiler requires the `override` keyword when we replace nonstatic virtual methods. It wants to know whether we were intending to override an existing method, so that if we weren't, it can warn us about naming collisions. (The absence of any equivalent keyword signifying the intention to implement an interface member is why the compiler cannot detect the same problem with default interface implementation. And the reason for this absence is that default interface implementation didn't exist prior to C# 8.0.)

We get a warning rather than an error, because the compiler provides a behavior that is likely to be safe when this situation has arisen due to the release of a new version of a library. The compiler guesses—correctly, in this case—that the developer who wrote the `CustomerDerived` type didn't mean to override the `LibraryBase` class's `Start` method. So rather than having the `CustomerDerived` type's `Start` method override the base class's virtual method, it *hides* it. A derived type is said to hide a member of a base class when it introduces a new member with the same name.

Hiding methods is quite different than overriding them. When hiding occurs, the base method is not replaced. [Example 6-31](#) shows how the hidden `Start` method remains available. It creates a `CustomerDerived` object and places a reference to that object in two variables of different types: one of type `CustomerDerived` and one of type `LibraryBase`. It then calls `Start` through each of these.

Example 6-31. Hidden versus virtual method

```
var d = new CustomerDerived();  
LibraryBase b = d;
```

```
d.Start();  
b.Start();
```

When we use the `d` variable, the call to `Start` ends up calling the derived type's `Start` method, the one that has hidden the base member. But the `b` variable's type is `LibraryBase`, so that invokes the base `Start` method. If `CustomerDerived` had overridden the base class's `Start` method instead of hiding it, both of those method calls would have invoked the override.

When name collisions occur because of a new library version, this hiding behavior is usually the right thing to do. If the customer's code has a variable of type `CustomerDerived`, then that code will want to invoke the `Start` method specific to that derived type. However, the compiler produces a warning because it doesn't know for certain that this is the reason for the problem. It might be that you *did* mean to override the method, and you just forgot to write the `override` keyword.

Like many developers, I don't like to see compiler warnings, and I try to avoid committing code that produces them. But what should you do if a new library version puts you in this situation? The best long-term solution is probably to change the name of the method in your derived class so that it doesn't clash with the method in the new version of the library. However, if you're up against a deadline, you may want a more expedient solution. So C# lets you declare that you know that there's a name clash and that you definitely want to hide the base member, not override it. As [Example 6-32](#) shows, you can use the `new` keyword to state that you're aware of the issue and definitely want to hide the base class member. The code will still behave in the same way, but you'll no longer get the warning, because you've assured the compiler that you know what's going on. But this is an issue you should fix at some point, because sooner or later the existence of two methods with the same name on the same type that mean different things is likely to cause confusion.

Example 6-32. Avoiding warnings when hiding members

```
public class CustomerDerived : LibraryBase
{
    public new void Start()
    {
        Console.WriteLine("Derived type's Start method");
    }
}
```

NOTE

C# does not let you use the `new` keyword to deal with the equivalent problem that arises with default interface implementations. There is no way to retain the default implementation supplied by an interface and also declare a public method with the same signature. This is slightly frustrating because it's possible at the binary level: it's the behavior you get if you do not recompile the code that implements an interface after adding a new member with a default implementation. You can still have separate implementations of, say, `ILibrary.Start` and `CustomerDerived.Start`, but you have to use explicit interface implementation.

Just occasionally, you may see the `new` keyword used in this way for reasons other than handling library versioning issues. For example, the `ISet<T>` interface that I showed in [Chapter 5](#) uses it to introduce a new `Add` method. `ISet<T>` derives from `ICollection<T>`, an interface that already provides an `Add` method, which takes an instance of `T` and has a `void` return type. `ISet<T>` makes a subtle change to this, shown in [Example 6-33](#).

Example 6-33. Hiding to change the signature

```
public interface ISet<T> : ICollection<T>
{
    new bool Add(T item);
    // ...other members omitted for clarity
}
```

The `ISet<T>` interface's `Add` method tells you whether the item you just added was already in the set, something the base `ICollection<T>` interface's `Add` method doesn't support. `ISet<T>` needs its `Add` to have a different return type—`bool` instead of `void`—so it defines `Add` with the `new` keyword to indicate that it should hide the `ICollection<T>` one. Both methods are still available—if you have two variables, one of type `ICollection<T>` and the other of type `ISet<T>`, both referring to the same object, you'll be able to access the `void Add` through the former and the `bool Add` through the latter.

Microsoft didn't have to do this. It could have called the new `Add` method something else—`AddIfNotPresent`, for example. But it's arguably less confusing just to have the one method name for adding things to a collection, particularly since you're free to ignore the return value, at which point the new `Add` looks indistinguishable from the old one. And most `ISet<T>` implementations will implement the `ICollection<T>.Add` method by calling straight through to the `ISet<T>.Add` method, so it makes sense that they have the same name.

Aside from the preceding example, so far I've discussed method hiding only in the context of compiling old code against a new version of a library. What happens if you have old code *already compiled* against an old library but that ends up *running* against a new version? That's a scenario you are highly likely to run into when the library in question is in the .NET runtime libraries. Suppose you are using third-party components that you have only in binary form (e.g., ones you've licensed from a company that does not supply source code). The supplier will have built these to use some particular version of .NET. If you upgrade your application to run with a new version of .NET, you might not be able to get hold of newer versions of the third-party components—maybe the vendor hasn't released them yet, or perhaps it has gone out of business.

If the components you're using were compiled for, say, .NET Standard 1.2, and you use them in a project built for .NET 8.0, all of those older components will end up using the .NET 8.0 versions of the runtime libraries. .NET has a versioning policy that arranges for all the components that a particular program uses to get the same version of the runtime libraries,

regardless of which version any individual component may have been built for. So it's entirely possible that some component, *OldControls.dll*, contains classes that derive from classes in .NET Standard 1.2, and that define members that collide with the names of members newly added in .NET 8.0.

This is more or less the same scenario as I described earlier, except that the code that was written for an older version of a library is not going to be recompiled. We're not going to get a compiler warning about hiding a method, because that would involve running the compiler, and we have only the binary for the relevant component. What happens now?

Fortunately, we don't need the old component to be recompiled. The C# compiler sets various flags in the compiled output for each method it compiles, indicating things like whether the method is virtual or not and whether the method was intended to override some method in the base class. When you put the `new` keyword on a method, the compiler sets a flag indicating that the method is not meant to override anything. The CLR calls this the *news*lot flag. When C# compiles a method such as the one in [Example 6-29](#), which does not specify either `override` or `new`, it also sets this same newslot flag for that method, because at the time the method was compiled, there was no method of the same name on the base class. As far as both the developer and the compiler were concerned, the `CustomerDerived` class's `Start` was written as a brand-new method that was not connected to anything on the base class.

So when this old component gets loaded in conjunction with a new version of the library defining the base class, the CLR can see what was intended—it can see that, as far as the author of the `CustomerDerived` class was concerned, `Start` is not meant to override anything. It therefore treats `CustomerDerived.Start` as a distinct method from `LibraryBase.Start`—it hides the base method just like it did when we were able to recompile.

By the way, everything I've said about virtual methods can also apply to properties, because a property's accessors are just methods. So you can define virtual properties, and derived classes can override or hide these

in exactly the same way as with methods. I won't be getting to events until [Chapter 9](#), but those are also methods in disguise, so they can also be virtual.

Static Virtual Methods

Before C# 11.0, you couldn't declare static methods as virtual, but as you saw in Chapters [3](#) and [4](#), interfaces can now do this. (Only interfaces, though. You still can't write `static virtual` or `static abstract` in any other kind of type.) [Example 6-34](#) shows an excerpt from the source for `INumberBase<T>` in the .NET runtime libraries. As you saw in [Chapter 4](#), this defines various characteristics common to numeric types, including a `static abstract` property, `One`.

Example 6-34. A static virtual property in an interface

```
public interface INumberBase<TSelf>
    : IAdditionOperators<TSelf, TSelf, TSelf>,
    ...
{
    /// <summary>Gets the value <c>1</c> for the type.</summary>
    static abstract TSelf One { get; }
    ...
}
```

This requires each numeric type to implement its own `One` property. (The binary representation of the number 1 is not the same across all numeric types.) But how do we ensure that we get the right implementation? Instance virtual method invocation selects the method based on the type of the object on which you invoke the method. But what about static virtual methods? Since there's no target object on which a static method is invoked, how is the runtime to know which type's implementation to use? With static virtual methods, the runtime has to use a different mechanism: the target type is determined by a generic type argument. As you saw in earlier chapters, we can invoke an interface's static virtual members only in generic code. If we constrain a type parameter to implement some interface that defines static virtual members, we can use those members through the type parameter, as [Example 6-35](#) shows.

Example 6-35. Invoking static virtual interface members

```
public static T Two<T>()  
    where T : INumberBase<T>  
{  
    return T.One + T.One;  
}
```

This `Two<T>` method returns the representation of the number 2 for any type that implements `INumberBase<T>`, relying on the fact that `INumberBase<T>` defines a static virtual property called `One`, along with a static virtual `+` operator. (It's not a very useful method, but it serves to illustrate how static virtual dispatch works.)

Since we're accessing this property with `T.One`, the specific implementation we get is determined by the type argument we supply for `T` when invoking this `Two<T>` method. When we write `Two<int>`, the CLR knows that `T.One` needs to invoke `int.One`. If we were to write `Two<Complex>`, it would invoke `Complex.One`. So the target of a virtual static invocation is determined at the point where we supply the generic type argument.

One consequence of this is that the invocation target for static virtual members becomes apparent earlier than it does for instance virtual members. This can sometimes make it easier for the CLR to optimize the code—when you instantiate a generic method with a value-typed type argument, the CLR usually generates code specific to that type, meaning that it can know exactly which method a static virtual invocation refers to when JIT (or AOT) compilation occurs, so it doesn't have to generate code that has to work out what to do at runtime. (There may be some cases where the CLR can do this for instance members too—if I write `"x".ToString()` there's no doubt that I'm invoking `string.ToString`, for example—but it often won't be able to.) The fact that virtual static method targets are known at JIT compile time for value types helps to enable code that exploits generic math to perform just as well as code that performs arithmetic more conventionally.

Default Constraints

The section [“Constraints”](#) described all but one of the ways in which you can define constraints for generic type parameters. It left the `default` constraint to this chapter, because that is used only in inheritance scenarios. It exists to deal with code like [Example 6-36](#).

Example 6-36. A base type with overloads distinguished only by a value type constraint

```
public class Base
{
    public virtual void F<T>(T? t) where T : struct { }
    public virtual void F<T>(T? t) { }
```

At first glance, this looks like it should not compile: this class defines two methods with the same name, and what would appear to be the same signature: they both return nothing, and both take an argument of `T?`. This seems ambiguous, but it's not, because the first method's `T` is constrained to be a `struct`. In general, the presence of a constraint is not enough to distinguish otherwise-identical method signatures for overloading purposes, but it is in this particular case. Since `T` is constrained to be a value type, `T?` means something different than it does for an unconstrained type. [Example 6-37](#) shows a different way to write the same code that makes it easier to see that these methods do have different signatures.

Example 6-37. Spelling out the effect of the `struct` constraint

```
public class BaseWithSignaturesMadeClear
{
    public virtual void F<T>(Nullable<T> t) where T : struct { }
    public virtual void F<T>(T? t) { }
```

That makes it possible for the compiler to distinguish between them.

[Example 6-38](#) shows various ways to call these two methods.

Example 6-38. Invoking the two methods

```
Base b = new();
int? nullInt = null;
int? nonNullableInt = 42;

// These call the 1st overload.
b.F(nullInt);
b.F(nonNullableInt);
b.F(default(int?));

// These call the 2nd overload.
b.F(42);
b.F("Hello");
b.F(default(int));

// This would cause a compiler error.
// b.F(null);
```

Types deriving from this class may be in for a surprise. Of the two `F` methods in `Base`, which would you expect [Example 6-39](#) to override? It has been declared in exactly the same way as the second one, in which the type argument is unconstrained, so you might expect it to override that. In fact, it overrides the first.

Example 6-39. Overriding one of two ambiguous-looking methods

```
public class DerivedOverrideStructMethod : Base
{
    public override void F<T>(T? t) { }
}
```

The reason for this is that in older versions of C# you weren't allowed to write [Example 6-36](#). It used to be illegal to write `T?` when `T` is an unconstrained type argument. (The rationale was that nullable value types work very differently than reference types, and since the compiler has to generate quite different code for each, it's not possible to support nullability unless `T` has been constrained so that the compiler knows whether it's a value type.) However, this turned out to be an onerous restriction, so

as a compromise, the compiler now allows it. The behavior with value types is slightly surprising—if you supply `int` as the argument for an unconstrained type parameter `T`, the effective type of `T?` is `int`, and not, as you might expect, `int?`—but it does at least mean things work as you’d expect for reference types.

C# allows us to omit the type constraint on overrides in cases where there’s no ambiguity (e.g., if the base defined only one `F` method, there would be absolutely no doubt as to which method [Example 6-39](#) means to override) because it would be annoying to have to duplicate the constraints. This means that in cases where the base class defines a single `F<T>` method with `T` constrained to be a `struct`, [Example 6-39](#) would unambiguously override that method, and would implicitly impose the same constraint.

Now imagine you’re using some library that defined such a `Base` type, with just the one `F` method, and you had written a class such as the one in [Example 6-39](#). Imagine now that some time later, the library gets updated to take advantage of the fact that we can now write `T?` for an unconstrained type, i.e., it changes the base class to look like [Example 6-36](#) by adding that second `F<T>` overload. We would not want the meaning of our code to be changed by this. Our override in [Example 6-39](#) has a first argument of type `T?`, but that’s really `Nullable<T>` because when we wrote it, it was unambiguously overriding the `struct`-constrained form, and that shouldn’t change just because a new version of the base class added a new overload.

To maintain backward compatibility, [Example 6-39](#) carries on meaning what it always meant. That’s how we end up with the slightly surprising result that given a base type like [Example 6-36](#), [Example 6-39](#) overrides the version of `F` where `T` is constrained to be a `struct`, and not the one it looks like it should override.

So what if you actually wanted to override the unconstrained method? This historical baggage means that the obvious syntax (not specifying a constraint in the override, matching the absence of a constraint in the base class) doesn’t work. And this is why the `default` constraint exists. It

is effectively a way to say explicitly that you mean to override the method where there are no constraints. [Example 6-40](#) shows how we can use this to override each method defined in [Example 6-36](#) independently.

Example 6-40. Using the `default` constraint

```
public class Derived : Base
{
    // This overrides the method with the "where T : struct" constraint
    public override void F<T>(T? t) { }

    // This overrides the method where T is unconstrained.
    public override void F<T>(T? t) where T : default { }
}
```

It's very rare to need to use this. Defining pairs of virtual methods like those in [Example 6-36](#) is likely to cause confusion and it is best avoided. The language needs to offer `default` constraints so that you don't end up with a `virtual` method that you simply can't override, but if you find yourself having to use it, it would be better to see if you can revisit your design to avoid this.

Sealed Methods and Classes

Virtual methods are deliberately open to modification through inheritance. A sealed method is the opposite—it is one that cannot be overridden. Methods are sealed by default in C#: methods cannot be overridden unless declared `virtual`. But when you override a virtual method, you can seal it, closing it off for further modification. [Example 6-41](#) uses this technique to provide a custom `ToString` implementation that cannot be further overridden by derived classes.

Example 6-41. A sealed method

```
public class FixedToString
{
```

```
public sealed override string ToString() => "Arf arf!";  
}
```

You can also seal an entire class, preventing anyone from deriving from it. As [Example 6-42](#) shows, you put `sealed` before the `class` keyword. You can also put it before the `record` keyword in a class record type.

Example 6-42. A sealed class

```
public sealed class EndOfTheLine  
{  
}
```

Some types are inherently sealed. Value types, for example, do not support inheritance, so structs, record structs, and enums are effectively sealed. The built-in `string` class is also sealed.

There are two normal reasons for sealing either classes or methods. One is that you want to guarantee some particular invariant, and if you leave your type open to modification, you will not be able to guarantee that invariant. For example, instances of the `string` type are immutable. The `string` type itself does not provide a way to modify an instance's value, and because nobody can derive from `string`, you can guarantee that if you have a reference of type `string`, you have a reference to an immutable object. This makes it safe for you to use in scenarios where you do not want the value to change—for example, when you use an object as a key to a dictionary (or anything else that relies on a hash code), you need the value not to change, because if the hash code changes while the item is in use as a key, the container will malfunction.

The other usual reason for leaving things sealed is that designing types that can successfully be modified through inheritance is hard, particularly if your type will be used outside of your own organization. Simply opening things up for modification is not sufficient—if you decide to make all your methods virtual, it might make it easy for people using your type to modify its behavior, but you will have made a rod for your own back when it comes to maintaining the base class. Unless you control all of the code that derives from your class, it will be almost impossible to

change anything in the base without breaking backward compatibility, because you will never know which methods may have been overridden in derived classes, making it hard to ensure that your class's internal state is consistent at all times. Developers writing derived types will doubtless do their best not to break things, but they will inevitably rely on aspects of your class's behavior that are undocumented. So in opening up every aspect of your class for modification through inheritance, you rob yourself of the freedom to change your class.

You should be very selective about which methods, if any, you make virtual. And you should also document whether callers are allowed to replace the method completely or whether they are required to call the base implementation as part of their override. Speaking of which, how do you do that?

Accessing Base Members

Everything that is in scope in a base class and is not private will also be in scope and accessible in a derived type. If you want to access some member of the base class, you typically just access it as if it were a normal member of your class. You can either access members through the `this` reference or just refer to them by name without qualification.

However, there are some situations in which you need to state explicitly that you mean to refer to a base class member. In particular, if you have overridden a method, calling that method by name will invoke your override recursively. If you want to call back to the original method that you overrode, there's a special keyword for that, shown in [Example 6-43](#).

Example 6-43. Calling the base method after overriding

```
public class CustomerDerived : LibraryBase
{
    public override void Start()
    {
        Console.WriteLine("CustomerDerived starting");
    }
}
```

```
        base.Start();  
    }  
}
```

By using the `base` keyword, we are opting out of the normal virtual method dispatch mechanism. If we had written just `Start()`, that would have been a recursive call, which would be undesirable here. By writing `base.Start()`, we get the method that would have been available on an instance of the base class, the method we overrode.

What if the inheritance chain is deeper? Suppose `CustomerDerived` derives from `IntermediateBase` and that `IntermediateBase` derives from `LibraryBase` and also overrides the `Start` method. In that case, writing `base.Start()` in our `CustomerDerived` type will call the override defined by `IntermediateBase`. There's no way to bypass that and call the original `LibraryBase.Start` directly.

In this example, I have called the base class's implementation after completing my work. C# does not care when you call the base—you could call it as the first thing the method does, as the last, or halfway through the method. You could even call it several times, or not at all. It is up to the author of the base class to document whether and when the base class implementation of the method should be called by an override.

You can use the `base` keyword for other members too, such as properties and events. However, access to base constructors works a bit differently.

Inheritance and Construction

Although a derived class inherits all the members of its base class, this does not mean the same thing for constructors as it does for everything else. Most public members of the base class will be public members of the derived class too, accessible to anyone who uses your derived class. Constructors are the exception: someone using your class cannot construct it by using one of the constructors defined by its base class.

There is a straightforward reason for this: if you want an instance of some type `D`, then you'll want it to be a full-fledged `D` with everything in it properly initialized. Suppose that `D` derives from `B`. If you were able to use one of `B`'s constructors directly, it wouldn't do anything to the parts specific to `D`. A base class's constructor won't know about any of the fields defined by a derived class, so it cannot initialize them. If you want a `D`, you'll need a constructor that knows how to initialize a `D`. So with a derived class, you can use only the constructors offered by that derived class, regardless of what constructors the base class might provide.

In the examples I've shown so far in this chapter, I've been able to ignore this because of the default constructor that C# provides. As you saw in [Chapter 3](#), if you don't write a constructor, C# writes one for you that takes no arguments. It does this for derived classes too, and the generated constructor will invoke the no-arguments constructor of the base class. But this changes if I start writing my own constructors. [Example 6-44](#) defines a pair of classes, where the base defines an explicit no-arguments constructor, and the derived class defines one that requires an argument.

Example 6-44. No default constructor in derived class

```
public class BaseWithZeroArgCtor
{
    public BaseWithZeroArgCtor()
    {
        Console.WriteLine("Base constructor");
    }
}

public class DerivedNoDefaultCtor : BaseWithZeroArgCtor
{
    public DerivedNoDefaultCtor(int i)
    {
        Console.WriteLine("Derived constructor");
    }
}
```

Because the base class has a zero-argument constructor, I can construct it with `new BaseWithZeroArgCtor()`. But I cannot do this with the derived

type: I can construct that only by passing an argument—for example, `new DerivedNoDefaultCtor(123)`. So as far as the publicly visible API of `DerivedNoDefaultCtor` is concerned, the derived class appears not to have inherited its base class's constructor.

In fact, it has inherited it, as you can see by looking at the output you get if you construct an instance of the derived type:

```
Base constructor
Derived constructor
```

When constructing an instance of `DerivedNoDefaultCtor`, the base class's constructor runs immediately before the derived class's constructor. Since the base constructor ran, clearly it was present. All of the base class's constructors are available to a derived type, but they can be invoked only by constructors in the derived class. [Example 6-44](#) invoked the base constructor implicitly: all constructors are required to invoke a constructor on their base class, and if you don't specify which to invoke, the compiler invokes the base's zero-argument constructor for you.

What if the base doesn't define a parameterless constructor? In that case, you'll get a compiler error if you derive a class that does not specify which constructor to call. [Example 6-45](#) shows a base class without a zero-argument constructor. (The presence of explicit constructors disables the compiler's normal generation of a default constructor, and since this base class supplies only a constructor that takes arguments, this means there is no zero-argument constructor.) It also shows a derived class with two constructors, both of which call into the base constructor explicitly, using the `base` keyword.

Example 6-45. Invoking a base constructor explicitly

```
public class BaseNoDefaultCtor
{
    public BaseNoDefaultCtor(int i)
    {
        Console.WriteLine($"Base constructor: {i}");
    }
}
```

```

    }
}

public class DerivedCallingBaseCtor : BaseNoDefaultCtor
{
    public DerivedCallingBaseCtor()
        : base(123)
    {
        Console.WriteLine("Derived constructor (default)");
    }

    public DerivedCallingBaseCtor(int i)
        : base(i)
    {
        Console.WriteLine($"Derived constructor: {i}");
    }
}

```

The derived class here decides to supply a parameterless constructor even though the base class doesn't have one—it supplies a constant value for the argument the base requires. The second just passes its argument through to the base.

Primary Constructors

If a base class has a primary constructor, this doesn't change anything for classes that derive from it. As [Example 6-46](#) shows, we can invoke a base class's primary constructor in exactly the same way as we would any other base class constructor.

Example 6-46. Invoking a base class's default constructor explicitly

```

public class BasePrimaryCtor(int i)
{
    public override ToString() => $"Base {i}";
}

public class DerivedCallingBasePrimaryCtor : BasePrimaryCtor
{
    public DerivedCallingBasePrimaryCtor()

```

```
        : base(123)
    {
    }
}
```

But what if the derived class has a primary constructor? [Example 6-1](#) already showed how this looks when using the base class's no-arguments constructor, but what if we need to pass arguments to a base constructor? In that case, we can supply an argument list after the base class name, as [Example 6-47](#) shows.

Example 6-47. Invoking a base constructor from a primary constructor

```
public class DerivedWithPrimaryCallingBaseCtor(int i) : BaseNoDefaultCtor(i)
{
}
```

A type with a primary constructor can use this syntax to invoke any base class constructor requiring arguments. It doesn't matter whether the base class constructor is a primary constructor or an ordinary one.

In some cases, you might not want to pass a constructor argument through like this: sometimes a derived type will know what value to pass to the base record type, as [Example 6-48](#) shows. (Incidentally, this also shows that record types can participate in inheritance. The syntax is the same as for ordinary classes.)

Example 6-48. Passing a constant to a base constructor

```
public abstract record Colorful(string Color);

public record FordModelT() : Colorful("Black");
```

Although the base `Colorful` record has a primary constructor requiring the `Color` property to be supplied, this derived type does not impose the same requirement. The popular story is that Ford's early car, the Model T, was only available in one color, so this particular derived type can just set

the `Color` itself. Users of the `FordModelT` record do not need to supply the `Color`, even though it's a mandatory argument for the base `Colorful` type. Pedants will by now be itching to point out that this paint constraint applied only for 12 of the 19 years for which the Model T was produced. I would draw their attention to [Example 6-49](#), which shows that although the `FordModelT` type does not require the `Color` property to be passed during construction, it can still be set with an object initializer. So this type enables the color to be specified just as it could with early and late Model Ts, but the default is aligned with the fact that the overwhelming majority of these cars were indeed black.

Example 6-49. Using a derived record that has made a mandatory base property optional

```
var commonModelT = new FordModelT();  
var lateModelT = new FordModelT { Color = "Green" };
```

The syntax shown in Examples [6-47](#) and [6-48](#), where we put the arguments for the base constructor directly after the base type's name, is available only on types that define a primary constructor. If you look closely at [Example 6-48](#), you'll see that after the `FordModelT` type name, there's an empty argument list, meaning that there is a no-arguments primary constructor. Although this may seem redundant, without it, we wouldn't be allowed to write `Colorful("Black")` after the colon. (We could have written an ordinary constructor instead, but that would have been more verbose.)

Developers often ask: *How do I provide all the same constructors as my base class, just passing the arguments straight through?* As you've now seen, the answer is: *write all the constructors by hand*. There is no way to get the C# compiler to generate constructors in a derived class that look identical to the ones that the base class offers. You need to do it the long-winded way. At least Visual Studio, VS Code, or JetBrains Rider can generate the code for you—if you click on a class declaration, and then click the Quick Actions icon that appears, it will offer to generate constructors with the same arguments as any nonprivate constructor in the base class, automatically passing all the arguments through for you. Even with such help,

this can become onerous, especially in types with large numbers of properties that require values during construction.

Mandatory Properties

If a type defines properties that must always be supplied with values during construction, it can enforce this by defining one or more constructors that require their values as arguments. But as you saw in [Chapter 3](#), C# 11.0 introduced an alternative: if we use the `required` keyword in a property declaration, this indicates that the property must be set even though there's no corresponding constructor argument. (We set it with the object initializer syntax.) The `required` keyword can be particularly useful when using inheritance because it can save us from repeating constructor parameter lists over and over. [Example 6-50](#) shows how that sort of problem can start.

Example 6-50. Inheriting mandatory properties set with a constructor

```
public record PropertiesInCtor(int Id, string Name, double Width);

public record MoreCtorProps(int Id, string Name, double Width, int X)
    : PropertiesInCtor(Id, Name, Width);

public record YetMore(int Id, string Name, double Width, int X, int Y)
    : MoreCtorProps(Id, Name, Width, X);

public record EvenMore(int Id, string Name, double Width, int X, int Y, int Z)
    : YetMore(Id, Name, Width, X, Y);
```

This is an inheritance hierarchy with four types. (I'm using records because they provide the easiest way to define a type with properties that must be supplied to a constructor. A `class` can have same problem; it would just be a larger example.) As we get further down the inheritance chain, the constructor parameter list gets longer and longer because it has to include all of the base type's parameters, even though each derived type adds just one extra property. I've had to declare the base class's three arguments four times over, and then repeat their names another three

times to pass them on to the base constructor—Isn't inheritance supposed to help me avoid duplicating code? If the base `PropertiesInCtor` class had many more primary constructor arguments, this could become seriously inconvenient. [Example 6-51](#) shows an alternative approach.

Example 6-51. Inheriting `required` properties

```
public record BaseWithManyRequiredProperties
{
    public required int Id { get; init; }
    public required string Name { get; init; }
    public required double Width { get; init; }
}

public record MoreRequiredProps : BaseWithManyRequiredProperties
{
    public required int X { get; init; }
}

public record YetMoreProps : MoreRequiredProps
{
    public required int Y { get; init; }
}

public record EvenMoreProps : MoreRequiredProps
{
    public required int Z { get; init; }
}
```

Admittedly, this has over twice as many lines of code, so you might not consider this to be an improvement. With this technique, you can't exploit record types' ability to generate properties automatically from a primary constructor (because we're trying to avoid constructors here). The upside is that the derived types no longer need to repeat the complete list of all properties. Constructors are inherited slightly differently from all other members, which is why we ended up repeating ourselves, but there's no such problem with properties. This wasn't a viable alternative before C# 11.0 added the `required` keyword because constructors used to be the only mechanism by which you could require particular properties to be supplied during construction. But now, if we construct an in-

stance of `EvenMoreProps`, the compiler will report an error unless we write an object initializer that sets all five properties.

Even in this example, where the relatively small number of properties on the base types mean the benefits are offset by the verbosity of ordinary property syntax, this approach still offers another advantage. It is much easier to see that each derived type adds just a single new property. In a type hierarchy where base types have many more properties than this, the primary constructor syntax becomes increasingly unwieldy, because every derived type needs to repeat all of the base's parameters *twice* (once in its constructor parameter list, and again to pass them to the base constructor). You can rapidly reach a point where the ability to omit all of that means that even with the extra verbosity of explicit properties, `required` properties still end up looking more succinct overall.

Field Initialization

As [Chapter 3](#) showed, a class's field initializers run before its constructor. The picture is more complicated once inheritance is involved, because there are multiple classes and multiple constructors. The easiest way to predict what will happen is to understand that although instance field initializers and constructors have separate syntax, C# ends up compiling all the initialization code for a particular class into the constructor. So a constructor performs the following steps: first, it runs field initializers specific to this class (so this step does not include base field initializers—the base class will take care of itself); next, it calls the base class constructor; and finally, it runs the body of the constructor. The upshot of this is that in a derived class, your instance field initializers will run before base class construction has occurred—not only before the base constructor body but even before the base's instance fields have been initialized.

[Example 6-52](#) illustrates this.

Example 6-52. Exploring construction order

```
public class BaseInit
{
    protected static int Init(string message)
```

```

    {
        Console.WriteLine(message);
        return 1;
    }

    private int b1 = Init("Base field b1");

    public BaseInit()
    {
        Init("Base constructor");
    }

    private int b2 = Init("Base field b2");
}

public class DerivedInit : BaseInit
{
    private int d1 = Init("Derived field d1");

    public DerivedInit()
    {
        Init("Derived constructor");
    }

    private int d2 = Init("Derived field d2");
}

```

I've put the field initializers on either side of the constructor just to show that their position relative to nonfield members is irrelevant. The order of the fields matters, but only with respect to one another. Constructing an instance of the `DerivedInit` class produces this output:

```

Derived field d1
Derived field d2
Base field b1
Base field b2
Base constructor
Derived constructor

```

This verifies that the derived type's field initializers run first, and then the base field initializers, followed by the base constructor body, and then

finally the derived constructor body. In other words, although constructor bodies start with the base class, instance field initialization happens in reverse.

That's why you don't get to invoke instance methods in field initializers. Static methods are available, but instance methods are not, because the class is a long way from being ready. It could be problematic if one of the derived type's field initializers were able to invoke a method on the base class, because the base class has performed no initialization at all at that point—not only has its constructor body not run, but its field initializers haven't run either. If instance methods were available during this phase, we'd have to write all of our code to be very defensive, because we could not assume that our fields contain anything useful.

As you can see, the constructor bodies run relatively late in the process, which is why we are allowed to invoke methods from them. But there's still potential danger here. What if the base class defines a virtual method and invokes that method on itself in its constructor? If the derived type overrides that, we'll be invoking the method before the derived type's constructor body has run. (Its field initializers will have run at that point, though. In fact, this is the main reason field initializers run in what seems to be reverse order—it means that derived classes have a way of performing some initialization before the base class's constructor has a chance to invoke a virtual method.) If you're familiar with C++, you might hazard a guess that when the base constructor invokes a virtual method, it'll run the base implementation. But C# does it differently: a base class's constructor will invoke the derived class's override in that case. This is not necessarily a problem, and it can occasionally be useful, but it means you need to think carefully and document your assumptions clearly if you want your object to invoke virtual methods on itself during construction.

Record Types

When you define a `record` type (or you use the more explicit but functionally identical `record class` syntax), the resulting record type is, from the runtime's perspective, still a class. Record types can do most of

the things that normal classes can—although they’re typically all about the properties, you can add other members such as methods and constructors (either primary or conventional). And as you saw in [Example 6-48](#), class-based records also support inheritance. (Since `record struct` types are value types, those do not support inheritance.)

There are some constraints on inheritance with record types. An ordinary class is not allowed to inherit from a record type—only record types can derive from record types. Similarly, a record type can inherit only from either another record type or the usual `object` base type. But within these constraints, inheritance with records works much as it does for classes.

When a record type defines a primary constructor, the compiler does a bit more work than it would for an ordinary class. Each constructor argument produces a property with the same name, and the compiler also generates a deconstructor. You are free to write an ordinary constructor in a record but you will no longer get either of these features. There’s one inheritance scenario in which you might want deconstruction, but you won’t be able to get the compiler to provide it for you. This happens when the base type has no primary constructor and defines a property explicitly. A derived type might want to make that mandatory by making it a constructor argument. You can do this, but you can’t write a primary constructor, because primary constructors can only pass arguments to base constructors—they can’t set base properties directly. You have to write the constructor in full, as [Example 6-53](#) shows.

Example 6-53. Making an optional base property class positional

```
public abstract record OptionallyLabeled
{
    public string? Label { get; init; }
}

public record LabeledDemographic : OptionallyLabeled
{
    public LabeledDemographic(string label)
    {
        Label = label;
    }
}
```

```

    }

    public void Deconstruct(out string? label) => label = Label;
}

```

Although this is not a primary constructor, it has a similar effect. The presence of the constructor in [Example 6-53](#) will prevent the compiler from generating a default zero-argument constructor, meaning that code using `LabeledDemographic` will be obliged to provide the `Label` property during construction, just as if it had a primary constructor. You automatically get a deconstructor when a record type has a primary constructor, but I’ve had to write my own here. The compiler doesn’t generate one because deconstruction ends up being a little odd when attempting to impose positional behavior in a type deriving from a nonpositional record (i.e., one without a primary constructor). The base class defines `Label` as optional, and even though we’ve defined a constructor that requires a non-null argument, it would be possible to follow the constructor with an object initializer that sets it back to `null`. (That would be weird but not illegal.) So our deconstructor ends up not quite matching our constructor—they specify different nullability.

Records, Inheritance, and the `with` Keyword

[Chapter 3](#) showed how you can create modified copies of record types using a `with` expression. This builds a new instance that has all the same properties as the original except for any new property values you specify in the braces following the `with` keyword. This mechanism has been designed with inheritance in mind: the instance produced by the `with` keyword will always have the same type as its input, even in cases where the code is written in terms of the base type, like [Example 6-54](#).

Example 6-54. Using `with` on a base record type

```

OptionallyLabeled Discount(OptionalyLabeled item)
{

```



```

    return item with
    {
        Label = "60% off!"
    };
}

```

This uses the abstract `OptionallyLabeled` record type from [Example 6-53](#). We can call this passing in any concrete type derived from that abstract base. [Example 6-55](#) calls it twice with two different types.

Example 6-55. Testing how `with` interacts with inheritance

```

Console.WriteLine(Discount(new OptionallyLabeledItem()));
Console.WriteLine(Discount(new Product("Sweater")));

```

Running that code produces this output:

```

OptionallyLabeledItem { Label = 60% off! }
Product { Label = 60% off!, Name = Sweater }

```

`Console.WriteLine` calls `ToString` on its input, and record types implement this by reporting their name and then their property values. So you can see from this that when the `Discount` method produced modified copies of its inputs, it successfully preserved the type. So even though `Discount` knows nothing about the `Product` record type or its `Name` property, when it created a copy with the new `Label` value, that `Name` property was correctly carried over.

This works because of code that the compiler generates for record types. I already described the copy constructor in [Chapter 3](#), but that alone would not make this possible—the `Discount` method doesn't know about the `OptionallyLabeledItem` or `Product` types, so it wouldn't know to invoke their copy constructors. So records also get a hidden `virtual` method with an unspeakable name, `<Clone>$`. The `with` expression in [Example 6-54](#) invokes this (before going on to set the `Label` property). The compiler-generated `<Clone>$` method invokes its own copy constructor. Since derived record types override `<Clone>$`, a `with` expres-

sion will always get a full copy of the input record no matter what its type is, even when the code is written in terms of a base type.

Special Base Types

The .NET runtime libraries define a few base types that have special significance in C#. The most obvious is `System.Object`, which I've already described in some detail.

There's also `System.ValueType`. This is the abstract base type of all value types, so any `struct` or `record struct` you define—and also all of the built-in value types, such as `int` and `bool`—derive from `ValueType`. Ironically, `ValueType` itself is a reference type; only types that derive from `ValueType` are value types. Like most types, `ValueType` derives from `System.Object`. There is an obvious conceptual difficulty here: in general, derived classes are everything their base class is, plus whatever functionality they add. So, given that `object` and `ValueType` are both reference types, it may seem odd that types derived from `ValueType` are not. And for that matter, it's not obvious how an `object` variable can hold a reference to an instance of something that's not a reference type. I will resolve all of these issues in [Chapter 7](#).

C# does not permit you to write a type that derives explicitly from `ValueType`. If you want to write a type that derives from `ValueType`, that's what the `struct` keyword is for. You can declare a variable of type `ValueType`, but since the type doesn't define any public members, a `ValueType` reference doesn't enable anything you can't do with an `object` reference. The only observable difference is that with a variable of that type, you can assign instances of any value type into it but not instances of a reference type. Aside from that, it's identical to `object`. Consequently, it's fairly rare to see `ValueType` mentioned explicitly in C# code.

Enumeration types also all derive from a common abstract base type: `System.Enum`. Since enums are value types, you won't be surprised to find out that `Enum` derives from `ValueType`. As with `ValueType`, you

would never derive from `Enum` explicitly—you use the `enum` keyword for that. Unlike `ValueType`, `Enum` does add some useful members. For example, its static `GetValues` method returns an array of all the enumeration's values, while `GetNames` returns an array with all those values converted to strings. It also offers `Parse`, which converts from the string representation back to the enumeration value.

As [Chapter 5](#) described, arrays all derive from a common base class, `System.Array`, and you've already seen the features that offers.

The `System.Exception` base class is special: when you throw an exception, C# requires that the object you throw be of this type or a type that derives from it. (Exceptions are the topic of [Chapter 8](#).)

Delegate types all derive from a common base type, `System.MulticastDelegate`, which in turn derives from `System.Delegate`. I'll discuss these in [Chapter 9](#).

Those are all the base types that the CTS treats as being special. There's one more base type to which the C# compiler assigns particular significance, and that's `System.Attribute`. In [Chapter 1](#), I applied certain annotations to methods and classes to tell the unit test framework to treat them specially. These attributes all correspond to types, so when I applied the `[TestClass]` attribute to a class, I was using a type called `TestClassAttribute`. Types designed to be used as attributes are all required to derive from `System.Attribute`. Some of them are recognized by the compiler—for example, there are some that control the version numbers that the compiler puts into the file headers of the EXE and DLL files it produces. In [Chapter 14](#) I'll show all of this.

Summary

C# supports single implementation inheritance, and only with classes or reference type records—you cannot derive from a struct at all. However, interfaces can declare multiple bases, and a class can implement multiple interfaces. Implicit reference conversions exist from derived types to base types, and generic interfaces and delegates can choose to offer addi-

tional implicit reference conversions using either covariance or contravariance. All types derive from `System.Object`, guaranteeing that certain standard members are available on all variables. We saw how virtual methods allow derived classes to modify selected members of their bases, and how sealing can disable that. We also looked at the relationship between a derived type and its base when it comes to accessing members, and constructors in particular.

Our exploration of inheritance is complete, but it has raised some new issues, such as the relationship between value types and references and the role of finalizers. So, in the next chapter, I'll talk about the connection between references and an object's life cycle, along with the way the CLR bridges the gap between references and value types.

¹ More precisely, the same assembly, and also friend assemblies. [Chapter 12](#) describes assemblies.